

DESIGN SPECIFICATION · ACCOUNTING →
PURCHASING

Three quotes, one memory: purchasing as a system that *learns*.

A sub-module inside HydroNexis-AI Accounting for side-by-side supplier comparison, purchase history pulled from SOS Inventory, and a Nexi-driven RFQ outreach loop that keeps every reply so the next comparison is smarter than the last.

Module · Accounting

View ·

acct_purchasing

Branch ·

claude/accounting-purchasing-suppliers-5p94sz

Head · 7403d49

Doc rev · 2026-07-27

§ 1

Purpose & scope

What this sub-module is for, and what it deliberately isn't.

Every non-trivial purchase at AteretPaz is meant to be shopped across three suppliers. Today that shopping happens in email threads and paper quotes; nothing carries forward. The Purchasing sub-module is a place to do the comparison once, in one shape, and keep it — so the next time the same product comes up, the answer is already half-written.

- It handles:** side-by-side supplier comparison (unit / specs / quantity / cost / delivery / other), purchase history per product-supplier pair, a supplier ledger with activity status, and a Nexi-driven outreach flow that dispatches RFQs by email or SMS and logs the reply.
- It does not:** replace SOS Inventory as the system of record for POs or receipts, replace QuickBooks as the ledger of bills, or approve payments. It is a decision-support and audit layer that reads from those systems and writes intent — quotes requested, quotes received, awards made.

§ 2

Coverage of the handoff outline

Every screen, integration and security item from the initial developer outline — mapped to the current build.

The initial handoff outline lists **16 primary screens**, **5 core integrations** and **6 security requirements**. This section maps each one to what is built, what is partial, and what is roadmap.

Legend: ✓ **DONE** = built and merged on branch · 🟡 **PARTIAL** = data + logic exist, dedicated screen still to build · 🟠 **ROADMAP** = not yet built; unlock path noted.

2.1 Primary screens (16)

#	SCREEN	STATUS	WHERE IT LIVES / WHAT TO FINISH
1	Purchasing Mission Control	 PARTIAL	The Purchasing view header carries a live stats banner (comparisons, open, products, suppliers, spend) + integration strip (SOS/QB connection, sync counters). A dedicated “mission control” landing that folds in KPIs, alerts, and drill-through cards is the next step — reuse hnxDashboard render patterns.
2	Purchase Requests	 ROADMAP	A form for a requester to raise a purchase need before it becomes a comparison. Add a new tab “  Requests” storing <code>hydroPro_acct_purch_requests[{id, requester, product, qty, unit, needBy, reason, dept, status:new approved converted}]</code> . “Convert to comparison” button seeds a comparison card.
3	SOS Inventory Check	✓ DONE	Header banner reads <code>window.__hnxSos.getMeta()</code> live; comparison cards include a “SOS item hint” row from <code>_purchSosItemHint(product)</code> — preferred vendor, SKU, last purchase cost, on-hand. Purchase history import from <code>/sos/purchase-orders</code> via <code>_purchSyncFromSOS()</code> .
4	Supplier Discovery	✓ DONE	Nexi outreach modal + <code>_purchGatherAllSuppliers()</code> unions Inventory Master, Accounting Vendors, SOS items’ preferred vendors, past PO vendors, and past-outreach responders. Includes the geo-scoped radius filter (\$10). Web-found suppliers can be added inline and are persisted to Vendors master.
5	Supplier Qualification	 PARTIAL	Suppliers already carry a reliability “score” field on <code>hydroPro_inv_suppliers_v1</code> and Nexi shows a reply-rate chip. A dedicated qualification screen (docs on file, cert expiry, credit check, quality-audit score, blacklist flag) still to build — extend the vendor record with <code>qualification.{status, certs[], credit, docs[], nextReviewOn}</code> .
6	RFQ Builder	✓ DONE	Editable RFQ template with variable substitution (<code>{{product}}</code> <code>{{qty}}</code> <code>{{unit}}</code> <code>{{specs}}</code> <code>{{needBy}}</code> <code>{{supplierContact}}</code> <code>{{requester}}</code> <code>{{companyName}}</code>) in the outreach modal. “Save as default template” persists it in <code>hydroPro_acct_purch_rfq_template</code> .
7	Email / SMS Communication Center	✓ DONE	RFQ dispatch via EmailJS auto-send (silent) → <code>mailto:</code> fallback, and SMS via device <code>sms:</code> URL scheme (body ≤ 380 chars). Every dispatch appended to <code>hydroPro_acct_purch_outreach</code> . The  Nexi RFQ Log tab is the “center” — filter, preview body, mark responses.
8	Quotation Management	✓ DONE	On each  RFQ Log row: response dropdown (pending / quoted / declined / no-response), quoted unit cost, lead days, notes. Marking “quoted” with a unit price auto-

#	SCREEN	STATUS	WHERE IT LIVES / WHAT TO FINISH
			appends a soft quote-record to Purchase History (<code>source: "rfq-quote"</code>) so it surfaces in comparisons.
9	Quote Comparison	✓ DONE	Comparisons tab — up to 3 suppliers per product, side-by-side matrix (unit / specs / qty / unit cost / total / delivery / payment terms / warranty / notes), cheapest badge auto-computed, per-supplier NEW/EXISTING/HAS-HISTORY tagging, per-product history sub-row.
10	Negotiation Center	● PARTIAL	Replies come back into the RFQ Log where the user can update the quote. A dedicated thread view (versioned quotes, counter-offer template, expiry date, revisions log) is roadmap — extend outreach records with <code>revisions[]: [{ts, side, unitCost, deliveryDays, terms, notes}]</code> .
11	Approval Workflow	☐ ROADMAP	Awards are direct today. Add <code>approvalRules{smallLimit, mediumLimit, largeLimit, approverBySize}</code> on the base config; gate <code>_purchAward()</code> to insert an <code>approvalRequest</code> record when total $\geq$ threshold; render a “🔒 Pending approvals” tray for the approver(s).
12	Purchase Order	● PARTIAL	PO data is READ from SOS via <code>_purchSyncFromSOS()</code> and rendered as history. Writing a NEW PO back into SOS is roadmap — add a POST route on the hnx-sync worker (<code>/sos/purchase-orders POST</code>) and hook <code>_purchCreatePO(compId)</code> on award.
13	Receiving	● PARTIAL	Each SOS-imported history row already carries <code>qty + receivedQty</code> ; a “partial” status is preserved when the PO is not fully received. A dedicated receiving screen (per-PO checklist, defects / short-shipment log, three-way match indicator) still to build.
14	Supplier Evaluation	● PARTIAL	📊 Suppliers Ledger rolls up orders count, total spend, last purchase, activity status (ACTIVE / DORMANT / NEW). Reliability score exists on the base record. A scheduled quarterly-review page (delivery on-time %, defect rate, price stability, response speed) is roadmap.
15	Reports	● PARTIAL	CSV export of purchase history is wired (source, status, ref, receivedQty columns included). A dedicated Reports screen with saved views + charts (spend by supplier, spend by category, price variance vs baseline, reply-rate by supplier) is roadmap.
16	Security & Audit	● PARTIAL	Immutability of SOS-imported rows and of RFQ dispatch data is enforced in <code>_purchDeleteHistory()</code> and <code>_purchOutreachSetResponse()</code> . Every write stamps requester from <code>getCurrentUser()</code> . A dedicated audit trail viewer (who / what / when timeline, filterable) still to build. See §12 for encryption + MFA + role notes.

2.2 Core integrations (5)

INTEGRATION	STATUS	HOW IT WIRES IN
SOS Inventory	✓ DONE	Read via <code>window.__hnxApi("/sos/purchase-orders")</code> for PO history import; <code>window.__hnxSos.getItems()</code> for the item hint row + preferred vendor fallback; <code>window.__hnxSos.getMeta()</code> for connection state. Write-back (create PO) roadmap.
Email	✓ DONE	Silent send via existing EmailJS integration (<code>_sendViaEmailJs</code>). <code>mailto:</code> fallback when EmailJS not configured. Full body archived in the outreach log.
SMS	✓ DONE	Device <code>sms:</code> URL scheme (opens native composer). Silent auto-send is roadmap — Twilio-backed <code>/nexi/sms</code> worker route.
Accounting (QB)	⦿ PARTIAL	Read <code>hydroPro_acct_qb_v1</code> for connection-state hint only. Bill-line ingest is roadmap — add a Make scenario for Bills-by-Vendor-with-Lines feeding <code>_purchSyncFromQB()</code> . Comparisons + PO awards will eventually push bill drafts back.
AI Recommendation Engine	✓ DONE	Registered as a Nexi domain via <code>registerNexiDomain()</code> . Answers ranked-supplier questions (“who supplies X”, “best price on X”, “suppliers near me”, “supplier reply rate”, “how many RFQs sent”) over live stored data. See §9.

2.3 Security & audit (6)

REQUIREMENT	STATUS	HOW IT IS MET
Role-based access	✓ DONE	The Purchasing sub-tab sits inside Accounting and inherits the module's permission gate via <code>VIEW_TO_MODULE / can()</code> . The Accounting role has full manage on this domain; other roles get view-or-none per the existing PERMS engine.
MFA	📅 ROADMAP	Session / MFA is app-wide (currently handled by the cloud-login worker for staff sign-in). No purchasing-specific MFA prompt exists on award/send — recommended to add a step-up MFA prompt before <code>_purchAward()</code> when order value > large-threshold.
Audit trail	🕒 PARTIAL	Every write carries <code>createdAt / updatedAt / updatedBy</code> where the app already has that pattern. Outreach rows are immutable after dispatch (only response fields editable). Dedicated audit viewer TBD — reuse the existing <code>hydroPro_audit_log</code> renderer.
Encryption	📅 ROADMAP	<code>localStorage</code> is not encrypted at rest. For sensitive supplier data recommend either (a) migrating to the app's existing IndexedDB store with a per-user derived key, or (b) proxying reads through the hnx-sync worker with encrypted server-side storage.
Approval limits	📅 ROADMAP	Same as primary #11. Config lives on the base object (<code>approvalRules</code>), gate lives on <code>_purchAward()</code> .
Supplier isolation	✓ DONE	Suppliers keyed by lowercased name; every collection filters by supplier. No supplier can see another's quote — the outreach log is per-supplier per-RFQ, dispatched one message per supplier via EmailJS / mailto / sms; no bcc / mass-send is ever used.

§ 3

Where it lives

Navigation, view registration, discovery.

- › **Primary view** · registered as `acct_purchasing`, dispatched by the app-wide `switchView()` the same way `acct_vendors`, `acct_customers`, `acct_invoices` and `acct_tax` are. Mounted lazily inside `#appMain`.
- › **Module attribution** · added to the `VIEW_TO_MODULE` map with value `accounting`, so the module header banner and permission checks resolve correctly.
- › **Deep-link from Accountant-AI overlay** · a new `purchasing` entry in `ACCT_TABS` shows a summary panel and a “Open Purchasing workspace” button that closes the overlay and calls `switchView('acct_purchasing')`.
- › **Direct URL / console** · `switchView('acct_purchasing')` from anywhere.

§ 4

Four tabs

One header, four working surfaces.

Header banner shows totals: comparisons, open comparisons, products tracked, suppliers used, total logged spend. Underneath, an integration strip: SOS connection state, QB cache state, history-row counts (SOS-imported vs manual), last-sync timestamp, and inline sync controls. Below the header sits the tab strip and the primary action  NEXI · FIND & CONTACT SUPPLIERS .

TAB	WHAT IT DOES	WRITES TO
 Comparisons	Add a product to shop for; capture up to three supplier quotes side by side; expand a card to see the matrix, the cheapest badge, per-supplier history, and a SOS item hint. Actions: award a winner, close, delete, log a purchase from a chosen quote, or ask Nexi for more quotes.	hydroPro_acct_purch_comparisons
 Purchase History	Flat log of every purchase — product, supplier, unit, qty, unit cost, total, PO ref, notes. Filterable search, CSV export. Rows imported from SOS show a SOS badge and cannot be manually deleted.	hydroPro_acct_purch_history
 Suppliers Ledger	Roll-up per supplier: orders count, distinct products, total qty, total spend, last purchase date, activity status (ACTIVE / DORMANT / NEW), recent products chips.	read-only rollup

TAB	WHAT IT DOES	WRITES TO
 Nexi RFQ Log	Every RFQ Nexi dispatched: sent-at, product, supplier, channel, response status, quoted unit cost, lead days, notes. Marking a row <i>quoted</i> auto-appends a soft quote-record to Purchase History so it appears in future comparisons.	hydroPro_acct_purch_outreach

§ 5

Data model

Every persistent shape, in one place.

Everything is stored in `localStorage` under the `hydroPro_*` prefix used across the app. Nothing is written to a backend without going through the existing hnx-sync worker (SOS pulls only).

KEY	SHAPE	PURPOSE
hydroPro_acct_purch_comparisons	array	Saved 3-supplier comparison sheets.
hydroPro_acct_purch_history	array	Merged purchase log — manual entries, SOS-imported PO lines, and RFQ-quote softlines.
hydroPro_acct_purch_outreach	array	RFQ send log with response tracking — the memory Nexi learns from.
hydroPro_acct_purch_sync_meta	object	SOS import status: {status, lastSync, poCount, added, updated}.
hydroPro_acct_purch_rfq_template	string	User-editable RFQ message template with variables.
hydroPro_acct_vendors	array	Existing Accounting Vendors list — Nexi reads it for contact info and writes web-found candidates back.
hydroPro_inv_suppliers_v1	array	Existing Inventory Master suppliers — richer contact data + reliability score.
hydroPro_sos_items_v1	array (IDB)	SOS items cache — read for preferred vendor and last purchase cost per SKU.
hydroPro_emailjs_config_v1	object	Existing EmailJS credentials — reused for silent RFQ auto-send.

Comparison record

```

{
  id: "pc_lqm7...",
  product: "Rockwool cube 100×100",
  category: "Growing media",
  needBy: "2026-08-14",
  requester: "EB",
  status: "open" | "awarded" | "closed",
  chosenSupplier: 0 | 1 | 2 | null,
  createdAt: 1721001600000,
  awardedAt: ...,
  suppliers: [           // up to 3 slots
    { name, unit, qty, specs, unitCost, deliveryDays, payTerms, warranty, notes }
  ]
}

```

Purchase-history record · a single row per PO line or per manual entry.

```

{
  id: "ph_sos_2026-21-P00528-01_0",
  source: "sos" | "manual" | "rfq-quote",
  poNumber: "2026-21-P00528-01", // sos only
  lineIdx: 0,                    // sos only – dedup key
  date: "2026-05-28",
  product: "Business Card 2×3.5",
  supplier: "Micropoint Graphics",
  sku: "",
  unit: "pc",
  qty: 700,
  receivedQty: 0,
  unitCost: 6.75,
  totalAmount: 4725,
  ref: "2026-21-P00528-01",
  status: "open" | "partial" | "closed",
  notes: "From SOS PO · open",
  importedAt: 1721001600000
}

```

Outreach record · one row per RFQ dispatched (per channel per supplier).

```
{
  id: "rfq_lqzmz..._0",
  sentAt: 1721001600000,
  product, qty, unit, specs, needBy, requester,
  supplier, supplierEmail, supplierPhone,
  subject: "RFQ · Rockwool cube 100×100",
  message: "Hello ...",           // fully-rendered body
  channel: "email" | "sms",
  delivery: "emailjs" | "mailto" | "sms-app" | "error",
  status: "sent" | "opened" | "error",    // dispatch result
  responseStatus: "pending" | "quoted" | "declined" | "no-response",
  responseAt: ...,
  quotedUnitCost: ...,
  quotedDeliveryDays: ...,
  responseNotes: "..."
}
```

§ 6

Comparison logic

How the matrix picks the cheapest and reads the history.

Cheapest badge · for each supplier column, `total = qty × unitCost`. Only columns with a positive total are eligible; the lowest wins the  **CHEAPEST** tag. Ties resolve to first-filled.

Award vs cheapest · the buyer's choice can differ from the cheapest — the column with the buyer's award gets a green top-border and a  **CHOSEN** action; the cheapest tag stays independently. When a comparison has no explicit award yet, the cheapest is used as the visual default.

NEW / EXISTING supplier badge · a supplier is *EXISTING* if their name appears in the merged history OR in the Vendors master (case-insensitive, trimmed). Otherwise *NEW*. This is computed at render time — no separate state.

Per-product history sub-row · `_purchLookupHistory(product, supplier)` filters merged history to matching product-supplier pairs and returns `{times, totalQty, totalCost, lastDate, rows[]}`. The card renders the last six line-items so the buyer sees drift in quantity and price without opening another view.

SOS item hint row · `_purchSosItemHint(product)` scans `window.__hnxSos.getItems()` for a name/sku match and returns the item's preferred vendor, SKU, last purchase cost and on-hand. Rendered as a single blue-tinted row between the criteria table and the history rows. If the supplier under review matches SOS's preferred vendor, an extra note surfaces on that column's history cell.

§ 7

Purchase history sources

Three streams merged into one log; every row is source-tagged.

SOS Inventory · `source: "sos"`

`_purchSyncFromSOS()` calls `/sos/purchase-orders` through the existing hnx-sync Cloudflare worker.

Every PO line becomes one history row. Dedup key is `poNumber + lineIdx`, so re-syncs update in place instead of duplicating.

Trigger: manual (button in header banner). Auto-import on first empty load can be added later.

Manual entry · `source: "manual"`

The History tab has an add form: product, supplier, date, unit, qty, unit cost, PO/ref, notes. Also written by “+ Log purchase” on a comparison card, using the chosen supplier’s quote.

Fully editable and deletable.

RFQ quotes · `source: "rfq-quote"`

When an outreach row is marked *quoted* with a unit price, a soft row is auto-appended: `qty:0, unitCost:<quote>`. It surfaces the price to future comparisons without pretending a purchase happened.

Suppresses double-counting because `qty*unitCost=0`.

QuickBooks · deferred

QB currently caches only trial-balance and aging in `hydroPro_acct_qb_v1`, not per-line vendor-item bills. The banner reads its connection state as a hint.

A `_purchSyncFromQB()` can be added in parallel, using the same `source:` dedup pattern; the rest of the UI needs no change.

GUARANTEE

Once `_purchSyncFromSOS()` has run, every card’s per-supplier “n× before · P× total · last DATE” row is real. The buyer never types history — they type comparisons.

§ 8

Nexi outreach flow

From “find suppliers” to logged RFQ, in five steps.

Entry points: the header primary button, or “🔍 Ask Nexi for more quotes” on any comparison card. Both open the outreach modal, prefilled with product / qty / unit / need-by from the card if launched from context.

Step 1

Gather candidates

`_purchGatherAllSuppliers(product)` unions Inventory Master, Accounting Vendors, SOS items’ preferred vendors, past PO vendors, and past-outreach responders.

Step 2

Rank & tag

Product-history match > has email > has phone > reliability score > alpha. Each row shows source chips, `EXISTING` / `NEW` / `HAS HISTORY`, and past reply-rate.

Step 3

User picks

Manual checkboxes, or one-click “top 3” / “existing”. A separate form lets the user add a supplier they found on the web — saved to Vendors master so Nexi remembers.

Step 4

Compose

Editable RFQ template with variable substitution: `{{product}}` `{{qty}}` `{{unit}}` `{{specs}}` `{{needBy}}` `{{supplierContact}}` `{{requester}}` `{{companyName}}`.

Step 5

Send & log

Email via existing EmailJS auto-send (silent) or `mailto:`. SMS via device `sms:` URL scheme with body ≤ 380 chars. Every dispatch appends one row to the outreach log with full body archived.

Channel semantics

CHANNEL	DELIVERY MODE	BEHAVIOR
 EMAIL	emailjs	Silent send via existing EmailJS integration. Status "sent". Requires the shared EmailJS config already used for report auto-send.
	mailto	Fallback when EmailJS is not configured. Opens the device mail client with recipient, subject, and body pre-filled. Status "opened" — the app can't verify actual send.
 SMS	sms-app	Opens the device SMS composer via <code>sms:</code> URL scheme. Body auto-shortened to 380 chars with an ellipsis. Status "opened".
	error	Dispatch threw. Full error stored in the row's <code>error</code> field.

§ 9

Nexi intelligence

The learning loop that makes tomorrow’s comparison start with an answer.

The purchasing sub-module registers a Nexi domain via `registerNexiDomain()`, using the app-wide domain registry pattern (Irrigation, Logistics, etc. all register the same way). The registration exposes the three collections and provides a custom `answer(qLower)` function.

ASK NEXI	WHAT SHE READS	WHAT SHE RETURNS
<i>“who supplies rockwool?”</i>	All supplier sources + history	Ranked list with contact info + past-order counts
<i>“best price on rockwool cubes”</i>	Purchase history (incl. RFQ quotes)	Cheapest supplier + next two, with reply-rate
<i>“supplier reply rate”</i>	Outreach log	Per-supplier quoted/sent scoreboard
<i>“how many RFQs have I sent”</i>	Outreach log	Totals + open comparisons + records

THE COMPOUNDING EFFECT

Every outreach is logged. Every quote reply is logged. Every awarded winner is logged. On the third comparison for the same product, Nexi’s ranking already knows: who quoted, who replied within a day, who was cheapest, who won last time. Manual entry is only ever for what hasn’t happened yet.

§ 10

Permissions & audit

Who sees, who edits, what's recorded.

- › The Accounting role has full `manageHR` and payroll access and read-only access to operational modules — the Purchasing sub-tab sits within Accounting and inherits the module's permission gate.
- › Each outreach row records `requester` from the current user profile (`getCurrentUser()`). Once an RFQ has been dispatched, its body and dispatch metadata are immutable; only the response fields (`responseStatus`, `quotedUnitCost`, `quotedDeliveryDays`, `responseNotes`) are editable.
- › SOS-imported history rows are protected from manual deletion — `_purchDeleteHistory()` explicitly refuses them and tells the user to delete the PO in SOS or use "Clear SOS-imported rows".
- › CSV export writes `source`, `status`, `ref`, and `receivedQty` alongside the numeric fields, so an offline audit can trace every row to its origin.

§ 11

Wiring points

Exactly which globals the sub-module touches.

GLOBAL / HOOK	READ?	WRITTEN?	PURPOSE
<code>switchView(v)</code>	hooked	—	Registers <code>acct_purchasing</code> in the dispatcher.
<code>VIEW_TO_MODULE</code>	—	merge	Adds <code>acct_purchasing: "accounting"</code> .
<code>ACCT_TABS</code>	—	splice	Adds the <code>purchasing</code> sub-tab to the Accountant-AI overlay dropdown.
<code>window.__hnxApi(...)</code>	call	—	Fetches <code>/sos/purchase-orders</code> for history import.
<code>window.__hnxSos.getItems()</code>	call	—	Reads cached SOS items for the item hint row.
<code>window.__hnxSos.getMeta()</code>	call	—	SOS connection state for the banner.
<code>_sendViaEmailJs(to, subj, body)</code>	call	—	Silent RFQ auto-send via the existing EmailJS integration.
<code>registerNexiDomain({...})</code>	call	—	Exposes the purchasing intelligence to Nexi's chat/brain.
<code>showToast(msg, tone)</code>	call	—	User feedback on sync / send / save.
<code>getCurrentUser()</code>	call	—	Requester name in the outreach signature.

§ 12

Limits & roadmap

What today's build deliberately doesn't do — and what unlocks each capability.

NOT YET	WHY	WHAT UNLOCKS IT
Automated web supplier search	A browser HTML app can't crawl external sites (CORS, no server).	Add <code>/nexi/supplier-search?q=</code> to the hnx-sync worker; the modal already has a slot for it.
Silent SMS auto-send	Browsers have no direct SMS API; <code>sms</code> : hands off to the device app.	Add a Twilio-backed <code>/nexi/sms</code> worker route; swap the <code>sms</code> : handoff for a POST.
QB bill-line import	Current QB cache is TB/aging only.	Add a Make scenario for <i>Bills by Vendor with Lines</i> ; feed <code>_purchSyncFromQB()</code> .
Automatic reply detection	Response state is human-set today.	An inbound-email webhook (Zapier/Make on your quotes inbox) → <code>/nexi/rfq-reply?rfqId=</code> → auto-flip <code>responseStatus</code> .
Multi-currency & VAT split	Amounts are single-currency, VAT-inclusive.	Add <code>currency</code> , <code>vatRate</code> , <code>vatMode</code> to history rows and quote fields; render alt-currency next to totals.
Approval workflow > PX	Awards are direct today.	Add a threshold rule + approver assignment on the comparison card, gated by role.
Purchase Order creation back into SOS	Only pulls from SOS, doesn't push.	SOS <code>/sos/purchase-orders</code> POST via a new worker route on award.

Nothing on this list breaks the current model. Each item extends a stored field, adds one worker route, or plugs a new record source into the same merged history log.

HydroNexis-AI · Accounting · Purchasing sub-module

Design revision 2026-07-27 · branch `claude/accounting-purchasing-suppliers-5p94sz`

DESIGN SPEC · PURCHASING · FEATURE
II · GEO-SCOPED SUPPLIER SEARCH

Small orders stay local. Big orders *travel farther.*

Nexi filters supplier suggestions by distance from a configurable base, and the search radius scales up with the estimated order value. The bigger the ₱, the wider she looks — because a large order justifies the transport a small one doesn't.

Module · Accounting → Purchasing

Feature · geo-scoped RFQ search

Branch · claude/accounting-purchasing-suppliers-5p94sz

Head · 4fac878

Doc rev · 2026-07-27

§ 1

The rule, in one sentence

Value determines geography.

A ₱4,000 pack of consumables shouldn't be sourced from another country. A ₱1.2M capex order shouldn't be limited to the barangay. The system encodes that judgement so Nexi doesn't need to be reminded, and every buyer follows the same rule.

25 km
100 km
500 km
2 000 km

The rings are the tiers. The base pin is your farm. Each ring is the radius unlocked by a higher order-value bucket. Nexi searches inside the smallest ring the order qualifies for, and widens only if it can't find enough candidates.

Rings are illustrative, not to scale — a real 2 000 km ring wouldn't fit next to a 25 km ring on the same page.

§ 2

Default tier ladder

Sensible starting values — every one editable.

UP TO ₱	RADIUS	MEANING · WHEN IT FIRES
10,000	25 km	Small · local only. Consumables, replacements, urgent top-ups. Freight would dwarf the order.
50,000	100 km	Regular · metro. Weekly resupply, mid-size components. Same-day / next-day trucking still economic.
200,000	500 km	Mid · regional. Bulk media, quarterly packaging runs. Willing to wait a few days for better price.
1,000,000	2,000 km	Large · national. Equipment, project-scale inputs. Whole-country search opens meaningful competition.
∞	∞	Very large · worldwide. Capex, imports, one-off large-ticket. International freight is a rounding error against the deal.

Ladder is edited in the settings modal: 📍 Location & radius. Rows can be added, removed, relabelled, and re-priced; the top row is always the ∞ catch-all. Ordering is by *maxValue* — the first row whose ceiling is ≥ the order value wins.

§ 3

How the value is inferred

The buyer types intent, not arithmetic.

The outreach modal exposes an `Est. value ₱` field. It's not required — Nexi guesses:

- › **From the SOS hint** · if the product matches a SOS item with a `purchaseCost`, estimated value = `qty × lastCost`. This is the primary path once SOS is connected.
- › **From past history** · if any past PO line has the same product name, the median past unit cost × current qty is used as a fallback (roadmap).
- › **Buyer override** · typing a number in the field wins over any guess. The Override km field wins over the tier the value picks. Rules serve the buyer, not the other way around.

§ 4

Auto-widen safety

Nexi never returns zero suppliers because of the geography rule.

Every tier has a floor: `minSuppliersForRadius` (default **3**). If the requested radius surfaces fewer than that many contactable suppliers, Nexi automatically promotes the search to the next larger tier and re-runs.

BEHAVIOUR ON WIDEN

The status strip in the modal flags the widen explicitly — “*auto-widened · fewer than min in the requested tier*” — so the buyer sees when the rule bent and can adjust the tier or the min-count without hunting.

The floor is per-tier by design; the config lives on the base-location object so it moves with the site.

§ 5

Distance computation

Great-circle, not driving distance.

`_purchDistanceKm(lat1, lng1, lat2, lng2)` uses the Haversine formula. It's the right choice for a geography filter: cheap, deterministic, no external service, correct within a fraction of a percent at country-scale distances.

It is *not* driving distance. For a “how far is the truck actually going” number, a future step could call a routing API — but for a filter that decides whether to invite a supplier to quote, great-circle is more than accurate enough, and one straight-line rule is easier to explain than a road network that changes.

§ 6

Base location

One pin. Editable. Optional browser geolocation.

Set once from the header button  Location & radius . The modal captures a display name, an address, latitude and longitude — and offers a  USE MY CURRENT LOCATION button that reads `navigator.geolocation` and fills the coords in place.

Default until edited: Manila metro at `14.5995, 120.9842` . That's a sensible pin for AteretPaz; other sites edit it once and forget it.

```
// hydroPro_acct_purch_base_location
{
  name: "AteretPaz farm",
  address: "",
  lat: 14.5995,
  lng: 120.9842,
  tiers: [
    { maxValue: 10000, radiusKm: 25, label: "Small · local only" },
    { maxValue: 50000, radiusKm: 100, label: "Regular · metro" },
    { maxValue: 200000, radiusKm: 500, label: "Mid · regional" },
    { maxValue: 1000000, radiusKm: 2000, label: "Large · national" },
    { maxValue: Infinity, radiusKm: null, label: "Very large · worldwide" }
  ],
  minSuppliersForRadius: 3,
  includeUnknown: true
}
```

§ 7

Supplier coordinates

Per-supplier lat / lng, with graceful fallback.

A dedicated key stores explicit coordinates per supplier — keyed by lowercased name so a rename in one place doesn't sever the link.

```
// hydroPro_acct_purch_supplier_locations
{
  "micropoint graphics": {
    lat: 14.6532,
    lng: 121.0498,
    address: "Quezon City",
    city: "Quezon City",
    country: "PH"
  },
  ...
}
```

Fallback path. If a supplier has no explicit coord record,

`_purchGetSupplierLocation()` reaches into the existing `hydroPro_inv_suppliers_v1` and `hydroPro_acct_vendors` stores and returns any address / city / country found there — so the row can still show something even if the distance chip has to say “no location · add”.

Adding coords in-flow. Every row in the outreach modal shows a  chip.

Rows without coordinates get a clickable `no location · add` chip that prompts for lat / lng inline; the answer is written straight into the map and the modal re-renders. No mode-switching to a settings page.

ROADMAP

Coord entry is manual today because the app can't geocode from the browser (no server-side routing service). A future `/nexi/geocode?address=` route on the hnx-sync worker would let Nexi auto-fill coords from any address on the vendor record.

§ 8

The outreach modal, changed

Two new controls, one new column, one status strip.

A
Est. value ₱ Auto-guessed from qty × SOS last-cost. Editable. Feeds the tier lookup.
B
Override km Any positive number wins over the tier. Blank means “use the tier”.
C
Status strip 🎯 Radius X km · tier Y · in-range vs outside count · widen flag · base name + coords.
D
📍 chip / row Every supplier row shows distance. In-range rows are full opacity, outside rows are dimmed.
E
Selectors Top 3 · Existing · All-in-range — every bulk selector respects the in-range flag.

Nothing is hidden by the filter — out-of-range rows are still visible, just dimmed and unchecked by the bulk selectors, so the buyer can override on a specific supplier without editing the tier ladder.

§ 9

Nexi can be asked directly

Same rule, natural-language surface.

ASK NEXI	WHAT SHE RETURNS
<i>“suppliers near me”</i>	Ranked list by distance from the base, top 5, with contact info and past-order counts.
<i>“local suppliers for rockwool”</i>	Same, filtered to the product word. Falls back to closest few if no one is inside the default radius.
<i>“suppliers within 50 km”</i>	Parses the number, honours the override, returns everyone inside that radius.
<i>“where is my base”</i>	Reads the base-location record — name and coords.

New keywords: radius , nearby , near me , local , distance , km , kilometer , area . They’re appended to the domain’s keyword array so the registry routes those questions to the geo answerer before falling through to the price / supplier answerers.

§ 10

Storage summary

Two keys added, everything else re-used.

KEY	PURPOSE
hydroPro_acct_purch_base_location	Base name + coords + tier ladder + auto-widen floor + unknown-inclusion flag.
hydroPro_acct_purch_supplier_locations	Map of supplier-lower → {lat, lng, address, city, country}.
hydroPro_inv_suppliers_v1	Read only for fallback address / city / country when a supplier has no explicit coords.
hydroPro_acct_vendors	Same fallback role as above.

`Infinity` can't survive a JSON round-trip, so on save it's written as `null` in the top tier's `maxValue`; on load the config-loader restores it. Any consumer that reads the config through `_purchGetBase()` never sees the sentinel.

Not doing yet

Extension points, called out.

DEFERRED	WHAT UNLOCKS IT
Geocode a plain address	<code>/nexi/geocode?address=</code> on the hnx-sync worker. Would auto-fill supplier coords from stored addresses on Vendors master.
Driving-distance / actual travel time	OSRM or Google Distance Matrix through the worker. Only worth adding if lead-time promises depend on it — otherwise great-circle is enough.
Freight-cost aware radius	Multiply <code>qty × unitCost</code> by a freight-per-km rate before the tier lookup, so the tier reflects <i>delivered</i> cost, not just <i>ordered</i> cost.
Per-category tier overrides	Attach a <code>categoryOverrides</code> map on the base — e.g. “perishables never go past 200 km, regardless of value”.
Multi-site (per-farm) bases	Promote the base record to an array keyed by site; the outreach modal picks the site the buyer selects.

None of the above breaks the current model. Each extension either adds a stored field, adds a worker route, or plugs a new signal into the same tier lookup.